

Strukturni Dizajn Paterni

1 Façade

Façade je obrazac dizajna koji pruža pojednostavljen interfejs prema složenom podsistemu.

U našem slučaju, možemo koristiti Façade za usklađivanje cijelog toka kviza, budući da pokretanje i završetak kviza uključuje koordinaciju više klasa odjednom: `KvizSesija`, `Pitanje`, `KvizSesijaPitanje` i `Leaderboard`.

- Kreiranje klase `KvizFacade` koja će definirati metode za pokretanje i završetak kviza unutar našeg sistema.
- Implementacija metoda `pokreniKviz()`, `završiKviz()` i `dohvatiRezultat()` koje interno koordiniraju sve potrebne klase podsistema.
- Unutar `KvizFacade` možemo sakriti svu složenost kreiranja sesije, nasumičnog odabira pitanja, bilježenja odgovora i ažuriranja leaderboarda.

Korištenje: Unutar sistema možemo koristiti `KvizFacade` umjesto direktnog pozivanja svake klase zasebno. Na taj način, naš sistem će biti neovisan o unutrašnjim detaljima podsistema kviza, olakšavajući buduće promjene i održavanje.

2 Proxy

Proxy je obrazac dizajna koji uvodi posredničku klasu između klijenta i stvarnog objekta, omogućavajući kontrolu pristupa i provjeru ovlaštenja bez izmjene originalnih klasa.

U našem slučaju, možemo koristiti Proxy za zaštitu osjetljivih operacija nad klasama `Pitanje` i `Feedback`, s obzirom na to da sistem ima tri korisničke uloge s različitim pravima pristupa.

- Definiranje klase `PitanjeProxy` koja će implementirati isti interfejs kao klasa `Pitanje` i sadržavati referencu na stvarni objekat.
- Implementacija provjere korisničke uloge prije izvođenja osjetljivih operacija poput `setTekst()`, koja je dozvoljena samo Administratoru i Moderatoru.
- Kreiranje klase `FeedbackProxy` koja kontrolira pristup metodama klase `Feedback` prema ulozi trenutno prijavljenog korisnika.

Korištenje: Umjesto direktnog pristupa klasama `Pitanje` i `Feedback`, kontroleri komuniciraju kroz odgovarajući Proxy. Na taj način, logika provjere ovlaštenja je centralizovana na jednom mjestu, a ne raspršena po cijelom sistemu.

3 Adapter

Adapter je obrazac dizajna koji omogućava saradnju između različitih interfejsa. U našem slučaju, možemo koristiti Adapter za integraciju vanjskog MAIL servisa koji se koristi za slanje obavijesti korisnicima.

- Kreiranje interfejsa `IObavijestServis` koji će definirati metodu `posaljiObavijest()` unutar našeg sistema.
- Implementacija konkretnog Adaptera `MailServisAdapter` koji će prilagoditi interfejs vanjskog mail servisa na interfejs definiran unutar sistema.
- Unutar konkretnog Adaptera možemo implementirati metode slanja poruka prema API-ju vanjskog mail servisa.

Korištenje: Unutar sistema možemo koristiti `IObavijestServis` za slanje obavijesti umjesto direktnog poziva vanjskog servisa. Na taj način, naš sistem će biti neovisan o konkretnim implementacijama vanjskih servisa, olakšavajući buduće promjene i održavanje.

4 Bridge

Bridge obrazac omogućava odvajanje apstrakcije od implementacije tako što uvodi posebne klase koje povezuju ove dvije komponente. Implementiranjem Bridge obrasca možemo stvoriti fleksibilan sistem u kojem logika generisanja kviza neće direktno ovisiti o konkretnom algoritmu odabira pitanja. Umjesto toga, koristit će se most koji će povezivati ove dvije komponente, omogućavajući zamjenu algoritma bez izmjene klijentskog koda koji pokreće kviz.

5 Composite

Naš sistem sadrži hijerarhijsku strukturu podataka: **Predmet** sadrži listu **Oblast** objekata, a svaka **Oblast** sadrži listu **Pitanje** objekata. Composite obrazac omogućava grupiranje objekata u hijerarhijske strukture kako bi se koristio pojednostavljen interfejs za rad sa složenim skupinama objekata.

Korištenjem Composite obrasca možemo stvoriti strukturu entiteta u našem sistemu tako da svaki entitet bude kompozitni objekt koji može sadržavati druge entitete ili pojedinačne objekte. Na primjer, predmet može sadržavati oblasti kao podobjekte, a oblast može imati više pitanja kao podobjekte.

6 Decorator

Decorator je obrazac dizajna koji omogućava dinamičko dodavanje ili promjenu funkcionalnosti objektima. Možemo koristiti Decorator za prilagodbu različitih funkcionalnosti prema korisničkim ulogama.

- Definiranje interfejsa `KorisnikFunkcionalnost` koji predstavlja osnovnu funkcionalnost korisnika.
- Implementacija `OsnovnaKorisnikFunkcionalnost` koji pruža osnovne funkcionalnosti korisnika.
- Kreiranje dekoratora `ModeratorDecorator` i `AdministratorDecorator` koji dodaju specifične funkcionalnosti prema ulozi.

Korištenjem ovog pristupa omogućavamo fleksibilnost sistema, omogućavajući lako dodavanje ili promjenu funkcionalnosti prema potrebama korisnika.